

Adaptive Spline Trajectory Planning with Simulated Annealing

1.0 Introduction

The goal of this project was to develop a custom path and motion planner to drive along curved [spline] paths. This project achieves this with a TurtleBot3, navigating a maze and crowded spaces in the ROS2-Gazebo Sim Environment. **Each spline path is trained to adapt and conform to its environment for robot safety when driving.**

2.0 Background

Our robot must navigate from a rest position to a target position. The environment is a PGM map, loaded from a Map Server node. The map is dilated and inflated to create a configuration space (c-space). The c-space is a discretized (vectorized) 2D graph, scaled to fit the continuous space the robot navigates. Each graph cell is either free or occupied by an obstacle or wall. Free spaces are places the robot can safely drive through.

Trajectory planning involves (1) path planning and (2) motion planning. The first is handled with heuristics and an A* graph search on the c-space between where the robot starts and its target position. The result is a connected chain of edges between free cells in the c-space. Common path planners, such as Point-to-Point (P2P), then parse a path from those free cells. However, common planners will follow the A* path closely to ensure robot safety. **This becomes a problem for spline planners.**

2.1 Project Motivation

Our robot creates paths to follow from quintic spline polynomials. Splines create organic and curved geometric paths. These are excellent for non-holonomic motion and smooth maneuvering. Our planner parses waypoints from an A* search to interpolate the spline path.

Splines take on several shapes and forms based on their curvature and the waypoints they interpolate. The A* search provides potential waypoints. But waypoints must be selectively chosen. As seen in **Figure 1**, a raw path is created from points connected by a dashed line, where the spline interpolates a subset of those points.

This becomes a balancing act. In our case, the A* path is the dashed line. Fewer waypoints means the spline path is smoother and can have larger maneuvers. But, it may be unclear what shape that spline takes on, and if it risks colliding with an obstacle. Conversely, using more waypoints constrains the spline onto the A* path that will ensure it does not collide with obstacles.

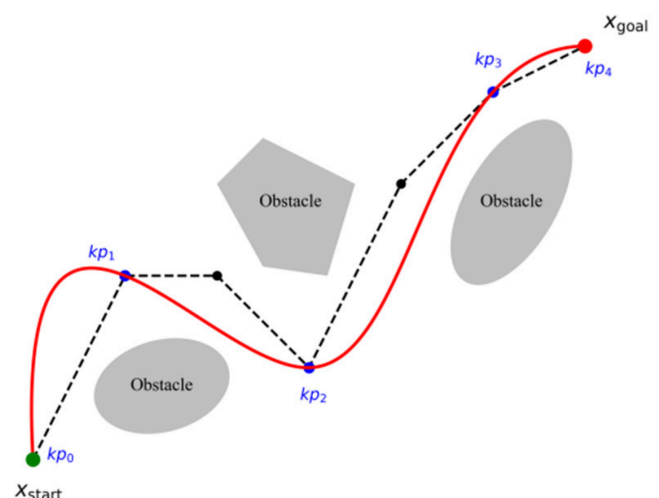


Figure 1: Illustrates spline path planning. [1]

2.2 Exploration-Exploitation Bottleneck on Splines and Safety

If too many waypoints are chosen from the A* path, then A* will have more influence over the path geometry than the curvature of the splines created. The balancing act from before becomes a trajectory bottleneck. The curvature and geometric malleability of splines is minimized when the number of waypoints and safety are maximized. **Figure 1** shows this trade-off, in which every other waypoint is rejected.

The spline path also follows the same homotopy as the original path. Viewing **Figure 1** from left to right, both the original and spline path travel over the first obstacle and weave under the second, and over the third in the same topological way. Although this allows for deterministic path planning, there are benefits to differing spline homotopy from the original path. For instance, consider if the spline in **Figure 1** could not pass through point **kp2**, and the spline instead traveled over (atop) the second obstacle from point **kp1** to **kp3**. **A spline path like that may turn out to be shorter or smoother.**

But giving splines the ability to explore different homotopies from the original path brings back the bottleneck of exploring at the expense of safety; this is especially true in environments that can change. **Therefore, adaptive spline planning that avoids obstacles could reconcile exploration beyond A* with safety.**

3.0 Motion Controls

Trajectory planning involves (1) path planning and (2) motion planning. This section will briefly cover the motion planning. Motion planning is based on robot odometry and the spline path it follows.

3.1 Motion Profiling with ICC

Given a path, the navigation node (**nav_node**) computes linear speeds. Linear speed is determined by acceleration and maximum-speed constraints. The linear speeds are calculated at each point along the discretized path by applying a trapezoidal motion profile.

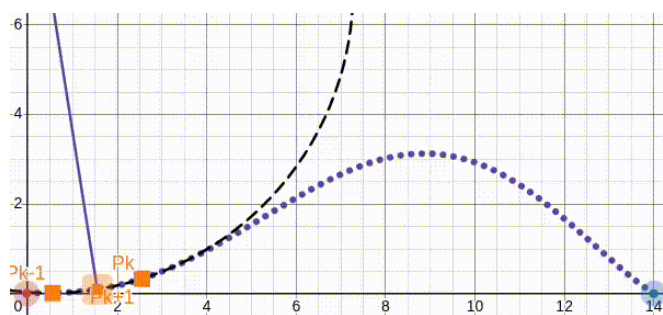


Figure 2: Illustrates approximating angular motion from spline path. [A custom Desmos demonstration] [\[2\]](#)

As seen in **Figure 2**, a path is approximated as circular motion with the instantaneous center of curvature (ICC). The robot looks ahead and behind its current position to approximate the local curvature of the circle.

The instantaneous angular velocity is then calculated from both the ICC and the motion-profiled linear speed. This leaves a discretized vector of linear and angular speeds to describe instantaneous velocity along each point of the path.

This serves as an excellent feed-forward controller for both finding base wheel speeds and base linear/angular speeds for publishing to the `/cmd_vel` topic.

3.2 Lateral PID Feedback Controller

The second half of motion is the feedback loop. Our robot uses a lateral PID controller that calculates error similarly to a Stanley controller.

As seen in **Figure 3**, positional and heading errors are used to calculate speed adjustments at position (c_x, c_y) along the path. Position (c_x, c_y) is the point along the path closest to the robot.

Although our robot used a PID controller, the error was modeled similarly to Stanley. Our robot tracked its closest unvisited point along the path based on the `/odom` topic. This point was placed in the robot's reference frame to measure the perpendicular [lateral] offset between it and the robot. This metric was an error for the PID controller that outputs a differential angular speed.

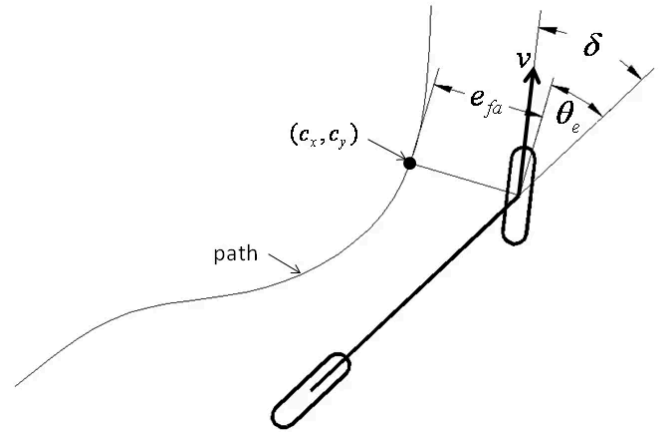


Figure 3: Illustrates error calculation for a Stanley Feedback Controller. [3]

Then, the motion profile computes wheel speeds for each point along the path before caching them. The closest unvisited point along the path both dictates what base speed is pulled from the cache in addition to the differential speed offset for drift correction. Together, the robot successfully followed spline paths.

4.0 Intelligent and Adaptive Splines

The remainder of this document covers the custom approach to adaptive spline planning. This section covers how splines can act as intelligent agents that adapt to avoid obstacles in the environment. **This is the central contribution of this project.** All work is implemented with Numpy.

4.1 Quintic Splines

Each path is made up of parametric quintic polynomials based on an input $\mathbf{t}$ with domain of $[0, 1]$. This function maps $\mathbf{t} \rightarrow (\mathbf{x}, \mathbf{y})$. A path is a set of poses $(\mathbf{x}, \mathbf{y}, \theta)$. A single spline interpolates between one pose to the next. If spline $\mathbf{S}$ interpolates from pose $\mathbf{P}$ to pose $\mathbf{P}'$, then functions $\mathbf{x}(\mathbf{t})$ and $\mathbf{y}(\mathbf{t})$ take the form:

$$\begin{aligned} x(t) &= a_{x0} + a_{x1}t + a_{x2}t^2 + a_{x3}t^3 + a_{x4}t^4 + a_{x5}t^5 \\ y(t) &= a_{y0} + a_{y1}t + a_{y2}t^2 + a_{y3}t^3 + a_{y4}t^4 + a_{y5}t^5 \end{aligned}$$

Equation 1: Models quintic spline $\mathbf{S}(t)$.

Both functions' coefficients are solved for with a time-parameter matrix Boundary Value Problem. The result is a spline $\mathbf{S}(\mathbf{t}) = (\mathbf{x}(\mathbf{t}), \mathbf{y}(\mathbf{t}))$, and its first and second derivative; computed via Horner's method. This is implemented in the `Quintic` class in `quintic.py`. Finally, our robot computes splines relatively between poses.

4.2 Cost-Optimization Problem

The spline exploration-to-safety bottleneck mentioned before is an optimization problem. Obstacle avoidance can be quantified in a cost metric in terms of spline parameters. These parameters then can be tuned by descending the cost map.

Solving for quintic spline coefficients requires knowing variables: x_0 , y_0 , θ_0 , x_1 , y_1 , θ_1 , k_0 , and k_1 . The first six are provided by the two poses the spline interpolates. The k_0 and k_1 are independent curvature constants to be set separately. **k_0 and k_1 are denoted as the 2D k-space.**

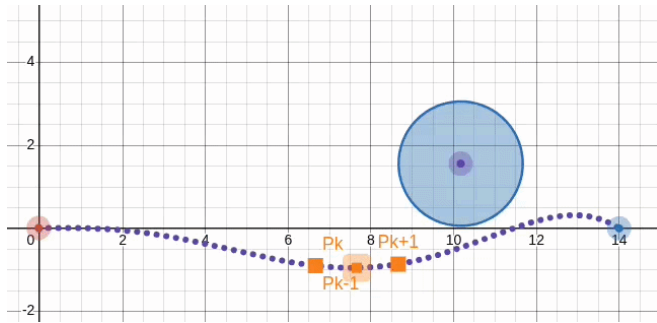


Figure 4: Illustrates a spline traversing the k-space.

[A custom Desmos demonstration] [4]

As seen in **Figure 4**, a spline is shown changing its curvature and shape based on the values for (k_0 , k_1). A subset of this k-space has the spline go over the blue circular obstacle, and another subset goes under the obstacle.

Our robot quantifies cost as overlap with known obstacles. This cost is minimized for obstacle avoidance.

4.3 Optimizing Splines with Simulated Annealing

4.4 Initial Tests

5.0 Trajectory Planning Package Pipeline

5.1 Waypoint Parsing

- A^* -> Parsing -> Centroid Snapping = Waypoints

5.2 Waypoint Dropout

5.3 Final Heading Tuning

5.4 ROS2 Package Design and Pipeline

6.0 Results

7.0 Conclusion

8.0 References

[1] Sun, Z., Luo, Q., Zhang, Z., Peng, Y., Liu, Q., Zheng, S., & Liu, J. (2025). An Integrated Path Planning and Tracking Framework Based on Adaptive Heuristic JPS and B-Spline Optimization. *Machines*, 13(8), 710.

<https://doi.org/10.3390/machines13080710>

[2] Papesch, M. (2026, June 12). Quintic spline demonstration. Desmos.com; Desmos.

<https://www.desmos.com/calculator/e7bd339d19>

[3] Kundu, S. (2020, July 19). Understanding Geometric Path Tracking Algorithms — Stanley Controller. Roboquest. <https://medium.com/roboquest/understanding-geometric-path-tracking-algorithms-stanley-controller-25da17bcc219>

[4] Papesh, M. (2026, June 15). Spline k-space demonstration. Desmos.com; Desmos. <https://www.desmos.com/calculator/hlemwksxi>